



Certificado profesional de Microsoft Python Development

Launch your career as a Python Developer.

Learn in-demand Python skills and become a job-ready developer in less than 4 months. No degree or experience required.



Instructor: [Microsoft](#)

Ir al curso

Ya estás inscrito

73.593 ya inscrito

Incluido con **coursera PLUS** • [Obtener más información](#)



Python Programming Fundamentals

CURSO 1 • 25 horas



Data Analysis and Visualization with Python

CURSO 2 • 22 horas



Automation and Scripting with Python

CURSO 3 • 25 horas



Web Development with Python

CURSO 4 • 23 horas



Advanced Python Development Techniques

CURSO 5 • 22 horas



Project Development in Python

CURSO 6 • 20 horas



Obtén un certificado profesional

Agregue esta credencial a su perfil de LinkedIn, currículum o CV. Compártala en redes sociales y en su evaluación de desempeño.

Plan PLATA 2026



Introducción a la Programación con Python

Aprende a hablar el idioma de las computadoras y da tus primeros pasos en el fascinante mundo del código.

Este material está diseñado para que cualquier persona, sin experiencia previa, comprenda los fundamentos de la programación y descubra por qué **Python** es el mejor punto de partida.

PROGRAMACIÓN

PYTHON

PRINCIPIANTES



¿Qué es la programación?

La idea central

Imagina poder hablar directamente con una computadora y decirle exactamente qué hacer.

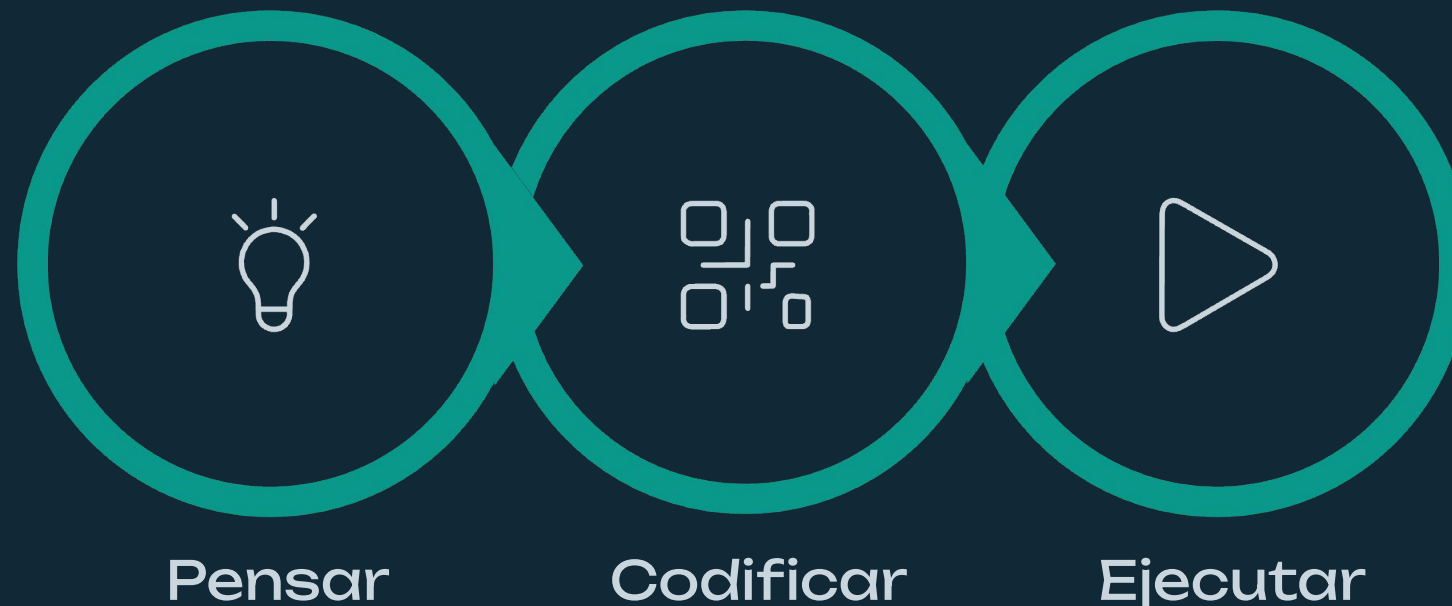
Eso es la **programación**: el arte de escribir instrucciones precisas que una máquina puede entender y ejecutar.

Estas instrucciones, llamadas **código**, actúan como un puente entre el pensamiento humano y la ejecución de la máquina.

Como una receta de cocina

En lugar de decir simplemente "**haz un pastel**", una receta proporciona ingredientes, cantidades y pasos concretos. La programación funciona igual:

1. **Lógica**: define qué debe hacer el programa
2. **Toma de decisiones**: ejecuta acciones distintas según condiciones
3. **Repetición**: repite instrucciones hasta lograr un objetivo
4. **Datos**: almacena y manipula información



Los lenguajes de programación

Así como las personas usan distintos idiomas para comunicarse, los programadores utilizan diferentes **lenguajes de programación**.

Cada uno tiene su propia **sintaxis** (reglas para escribir código) y **semántica** (significado del código).



Java

Robusto y ampliamente usado en **aplicaciones empresariales** y apps para Android.



C++

Alto rendimiento, usado en **videojuegos** y programación de sistemas operativos.



JavaScript

El lenguaje del navegador, imprescindible para el **desarrollo web** interactivo.



Python

Famoso por su **versatilidad y legibilidad**, ideal para principiantes y profesionales.

¿POR QUÉ PYTHON?

Python: simple, legible y poderoso

Python es un lenguaje de alto nivel diseñado para que su sintaxis se parezca al inglés natural. Esto permite a los principiantes concentrarse en la **lógica y resolución de problemas**, en lugar de batallar con una sintaxis compleja.

Python

```
length = 10
width = 5
area = length * width
print("Área:", area)
```

Claro, directo y legible. Refleja directamente la fórmula matemática.

C++ (mismo resultado)

```
#include <iostream>

int main() {
    int length = 10;
    int width = 5;
    int area = length * width;
    std::cout << "Área: " << area << std::endl;
    return 0;
}
```

Más verboso y con más requisitos de sintaxis para el mismo cálculo.

El ecosistema de Python

Una de las mayores fortalezas de Python es su enorme colección de **bibliotecas y frameworks**. Permiten realizar tareas complejas sin escribir todo desde cero, acelerando enormemente el desarrollo.

Análisis de datos

Pandas: trabaja con datos tabulares como hojas de cálculo

NumPy: cálculos numéricos y manejo de matrices multidimensionales

Desarrollo web

Django: framework completo con autenticación, bases de datos y rutas integradas

Flask: framework ligero que ofrece mayor flexibilidad y control

Inteligencia Artificial

Scikit-learn: algoritmos de aprendizaje automático (clasificación, regresión)

PyTorch: creación de modelos de aprendizaje profundo (deep learning)

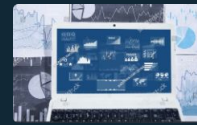
Python en el mundo real

Python no es solo un lenguaje de estudio: se usa a diario en empresas, universidades y proyectos de todo el mundo. Aquí algunos ejemplos concretos.



Análisis de redes sociales

Un analista accede a la API de una red social con Python, descarga miles de comentarios y usa bibliotecas de análisis de texto para estudiar la **opinión pública** sobre una marca en tiempo real.



Análisis financiero

Un analista usa **pyfinance** para descargar datos bursátiles históricos y genera automáticamente informes visuales de tendencias, ahorrando horas de trabajo manual.



Educación interactiva

Un profesor crea un cuestionario interactivo con **Tkinter**, construyendo una interfaz gráfica con ventanas, botones y etiquetas sin necesidad de herramientas externas.

Python vs. otros lenguajes para principiantes

El debate

Algunas personas argumentan que es mejor comenzar con lenguajes de **nivel más bajo** como C++, Java, C# o Rust, ya que ayudan a comprender mejor cómo funciona la computadora internamente: la memoria, los punteros y la gestión de recursos.

Sin embargo, estos lenguajes presentan una **curva de aprendizaje empinada** que puede desmotivar a quienes están empezando.

La ventaja de Python

Python permite aprender primero lo más importante:

Lógica de programación

- **Pensamiento computacional**

Resolución de problemas reales

Es como aprender a conducir en un **automático** antes de pasar a uno manual. Una vez que dominas la lógica, aprender otros lenguajes es mucho más sencillo.



Una comunidad global de apoyo

Python cuenta con una de las **comunidades más activas y acogedoras** del mundo de la programación. Esto significa que nunca aprenderás solo.

Documentación oficial

Extensa, bien organizada y disponible en varios idiomas. Incluye tutoriales para todos los niveles.

Tutoriales y cursos

Miles de recursos gratuitos en YouTube, plataformas como Coursera o edX, y sitios especializados.

Foros y comunidades

Stack Overflow, Reddit, grupos Telegram y otras redes ofrecen respuestas rápidas a cualquier duda, por pequeña que sea.

Proyectos abiertos

Miles de proyectos en GitHub donde puedes aprender leyendo código real y contribuyendo activamente.

Programación para Resolver Problemas

Los fundamentos de la programación te ayudan a **dividir problemas grandes en pasos pequeños y manejables**. Este enfoque lógico, analítico y estructurado es la esencia del pensamiento computacional.

Código eficiente

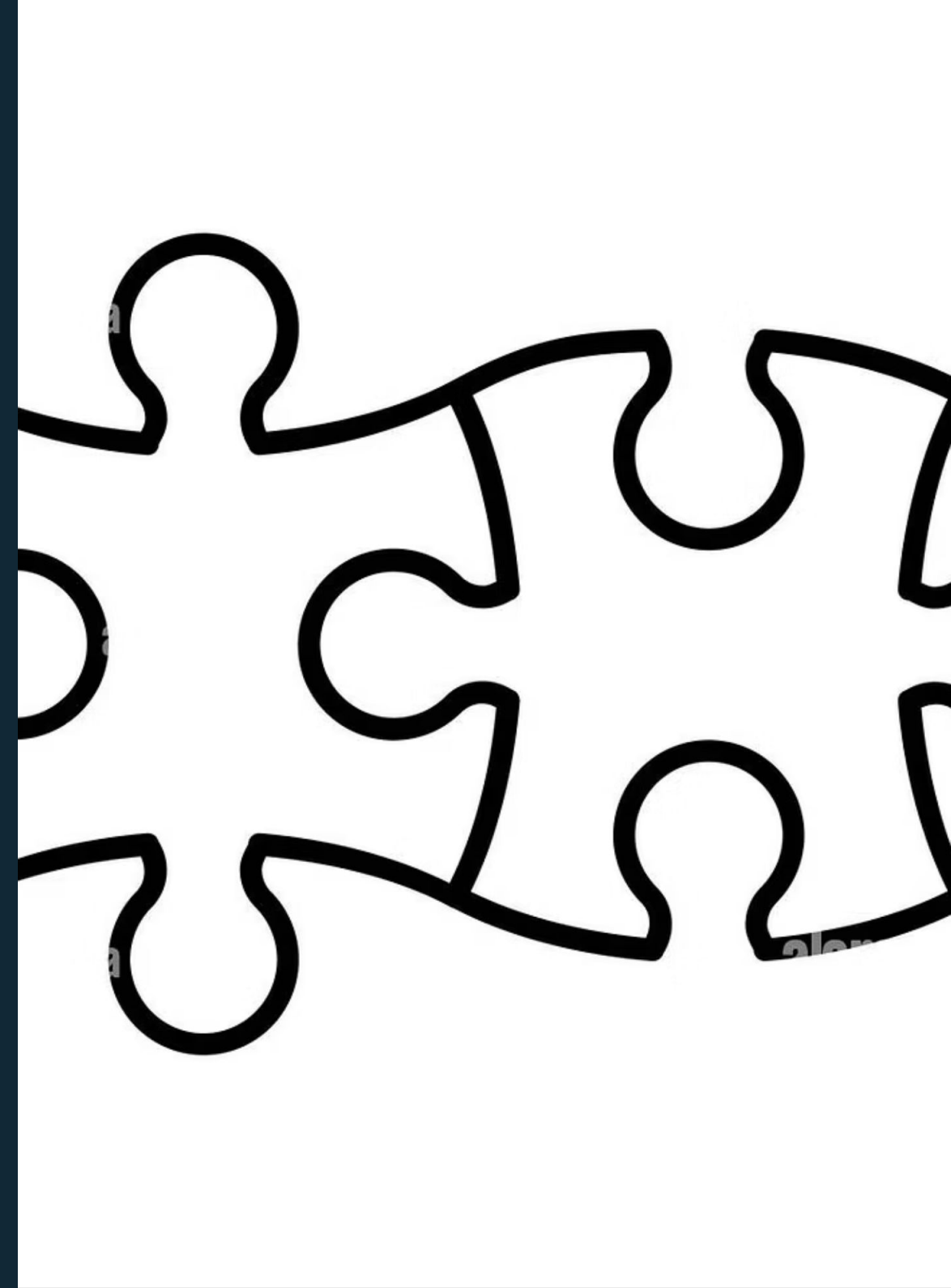
Soluciones que consumen menos recursos y procesan datos más rápido.

Código optimizado

Algoritmos bien diseñados que evitan pasos innecesarios.

Código escalable

Programas que pueden crecer y adaptarse a nuevas necesidades sin reescribirse.



Conceptos fundamentales: Variables y Datos

Para trabajar con información, todos los lenguajes de programación se apoyan en dos herramientas esenciales:
las **variables** y las **estructuras de datos**.

Variables

Actúan como **contenedores con nombre** que almacenan un valor. Ese valor puede cambiar durante la ejecución del programa.

Números: `edad = 25`

Texto: `nombre = "Ana"`

Decimales: `precio = 9.99`

Listas

Secuencias ordenadas de elementos del mismo o distinto tipo. Permiten agrupar datos relacionados.

- `colores = ["rojo", "azul"]`
- Se accede por posición (índice)
- Se pueden añadir o eliminar elementos

Diccionarios

Colecciones de pares **clave–valor** que permiten acceder a datos por nombre, no por posición.

- `alumno = {"nombre": "Luis", "nota": 9}`
- Las claves son únicas
- Ideales para representar objetos reales

Tipos de Datos

Los **tipos de datos** indican qué clase de información puede almacenar una variable. Usar el tipo correcto es esencial para evitar errores y garantizar que el programa funcione correctamente.

Enteros (*int*)

Números enteros sin decimales.

```
edad = 10  
temperatura = -5
```

Flotantes (*float*)

Números con parte decimal.

```
pi = 3.14  
precio = -2.5
```

Cadenas (*str*)

Texto entre comillas simples o dobles.

```
saludo = "Hola"  
ciudad = 'Lima'
```

Booleanos (*bool*)

Valores lógicos: verdadero o falso.

```
activo = True  
pagado = False
```

Operadores

Los **operadores** son símbolos que realizan operaciones sobre los datos. Permiten **manipular y transformar la información** dentro de un programa, desde cálculos matemáticos hasta decisiones lógicas complejas.

Aritméticos

Realizan operaciones matemáticas básicas.

+ Suma

- Resta

* Multiplicación

** Potencia

/ División

// División Entera

% Residuo o Modulo

De comparación

Comparan dos valores y devuelven un booleano.

> Mayor que

< Menor que

== Igual a

!= Distinde de

Lógicos

Combinan condiciones: **AND, OR, NOT**

📌 Ejemplo: la figura es azul y triangular → `color == "azul" and forma == "triángulo"`

Flujo de Control

El **flujo de control** determina el orden en que se ejecutan las instrucciones.

Gracias a él, un programa puede tomar decisiones y repetir acciones automáticamente.

Condicionales

Permiten ejecutar diferentes bloques de código según una condición.

Son la base de la toma de decisiones.

```
if compra > 100:
    descuento = 10%
elif compra > 50:
    descuento = 5%
else:
    descuento = 0%
```

Bucles

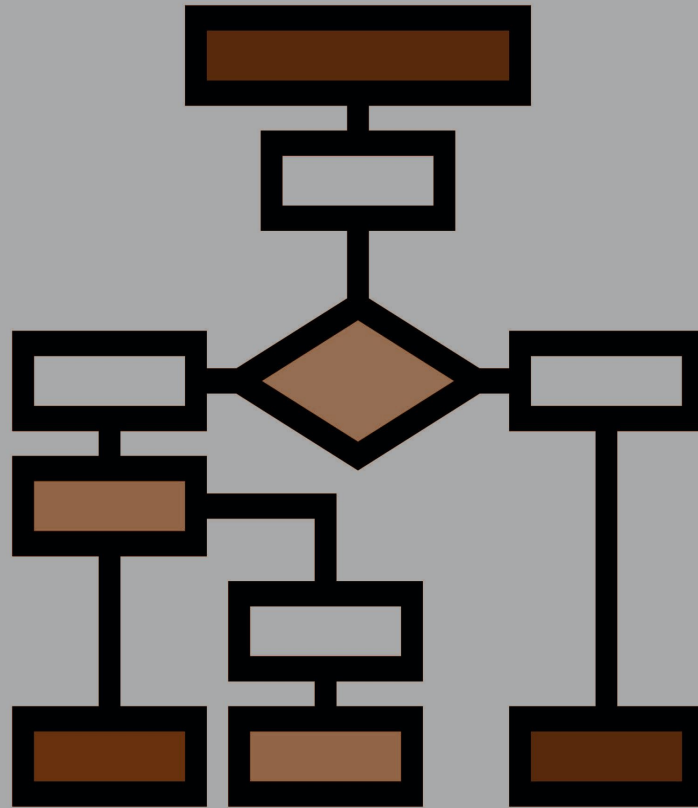
Repiten un bloque de código mientras se cumpla una condición o para cada elemento de una colección.

for: itera sobre una secuencia de elementos

```
for i in range(5):
    print(i)
```

while: repite mientras la condición sea verdadera

```
while saldo > 0:
    saldo -= 10
```



Funciones

Las **funciones** son bloques de código reutilizables que realizan una tarea específica. En lugar de escribir el mismo código varias veces, defines una función una sola vez y la llamas cuando la necesitas.

```
def saludar(nombre):  
    mensaje = "Hola, " + nombre  
    return mensaje
```

```
resultado = saludar("Ana")  
print(resultado)  
# Salida: Hola, Ana
```



Organización

Divide el código en bloques lógicos y manejables.



Reutilización

Escribe una vez, usa muchas veces en todo el programa.

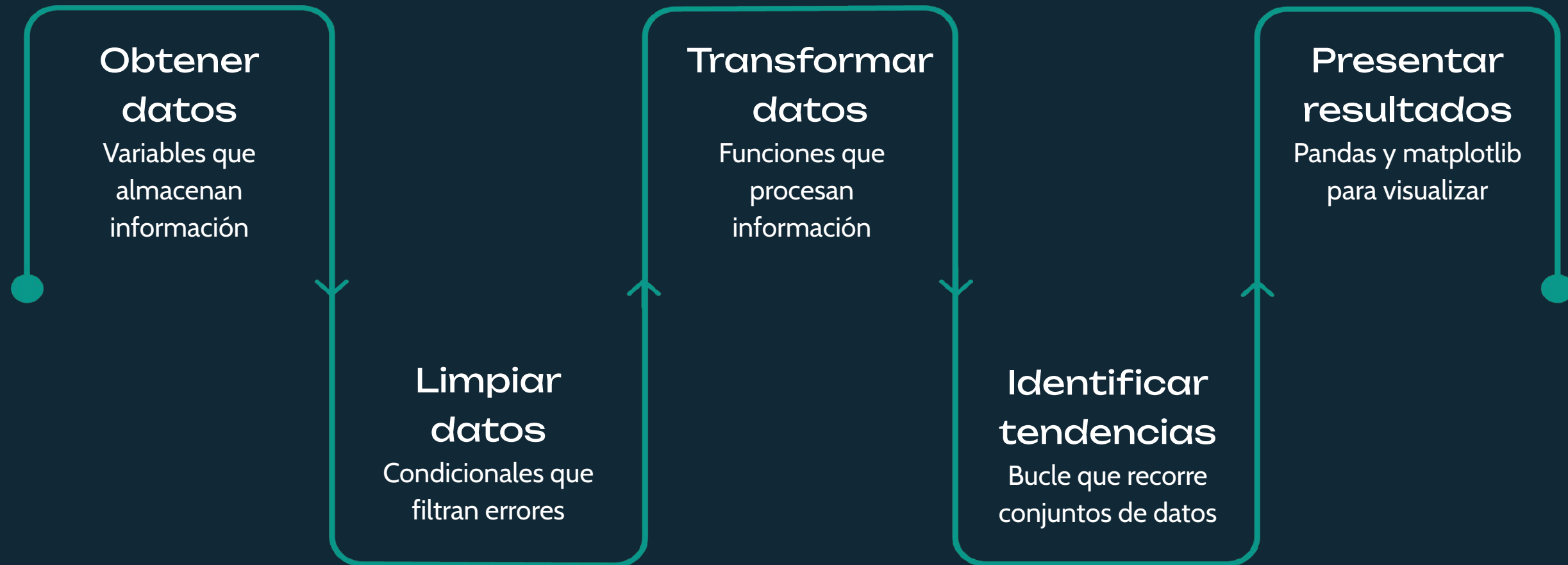


Mantenimiento

Corrige o mejora el código en un solo lugar.

Ejemplo: Análisis de Compras en una Tienda en Línea

Supongamos que debes crear un programa para **analizar las compras de clientes**. Observa cómo cada fundamento cumple un rol clave en el proceso:



Este flujo de trabajo ilustra perfectamente cómo **variables, bucles, condicionales y funciones** trabajan en conjunto para resolver un problema real del mundo empresarial.

